Entity–Relationship (ER) Diagrams

A teaching guide: core concepts, notation, and 10 worked real-world scenarios with diagrams

Contents

1. 1. What is an ER Diagram?
2. 2. Core Components
3. 3. Notation Legend (Chen's Notation)
4. Scenario 1 — Library Management
5. Scenario 2 — Hospital Management
6. Scenario 3 — University Enrollment
7. Scenario 4 — Company / Department / Dependents (Weak Entity)
8. Scenario 5 — Bank System
9. Scenario 6 — E-Commerce Ordering
10. Scenario 7 — Airline Reservation
11. Scenario 8 — School (Generalization/Specialization)
12. Scenario 9 — Food Delivery Platform
13. Scenario 10 — Hotel Reservation (Weak Entity + Aggregation)
14. Teaching Tips & Practice Exercise

1. What is an ER Diagram?

An **Entity-Relationship (ER) Diagram** is a conceptual, high-level blueprint of a database. It shows **what data** a system needs to store — the real-world "things" (entities), their properties (attributes), and how those things relate to each other (relationships) — *before* any table is created.

It is the bridge between a written requirement ("a library has books, members borrow books...") and a relational schema (tables, primary keys, foreign keys). Students should think of it as a design tool, not a drawing exercise: every symbol maps to a schema decision later.

2. Core Components

Component	Meaning	Example
Entity	A real-world object with independent existence, stored as a table	Student, Book, Doctor
Entity Set	Collection of entities of the same type	All Students
Attribute	A property of an entity → becomes a column	Name, Roll No
Key Attribute	Uniquely identifies each entity (underlined)	Student_ID
Composite Attribute	Made of sub-parts	Address = (Street, City, Zip)
Multivalued Attribute	Can hold more than one value (double oval)	Phone Numbers
Derived Attribute	Computed from another attribute (dashed oval)	Age from Date of Birth
Relationship	An association between two or more entities	Student *Enrolls In* Course
Cardinality	How many instances participate: 1:1, 1:N, M:N	One Department has Many Employees
Participation	Total (double line, mandatory) vs Partial (single line, optional)	Every Employee must belong to a Department
Weak Entity	Has no key of its own; depends on an owner entity (double rectangle)	Dependent (depends on Employee)
Identifying Relationship	Connects a weak entity to its owner (double diamond)	Employee *Has* Dependent
Generalization / Specialization (ISA)	A superclass split into subclasses that inherit attributes	Person → Student, Employee
Aggregation	Treats a whole relationship as a single higher-level entity	(Project–Employee Assignment) Approved-By Manager

3. Notation Legend (Chen's Notation)



Rectangle — Entity



Diamond — Relationship



Oval — Attribute



Dashed Oval — Derived Attribute

Single line = **Partial** participation | Double line = **Total** participation



Double Rectangle — Weak Entity



Double Diamond — Identifying Relationship



Double Oval — Multivalued Attribute

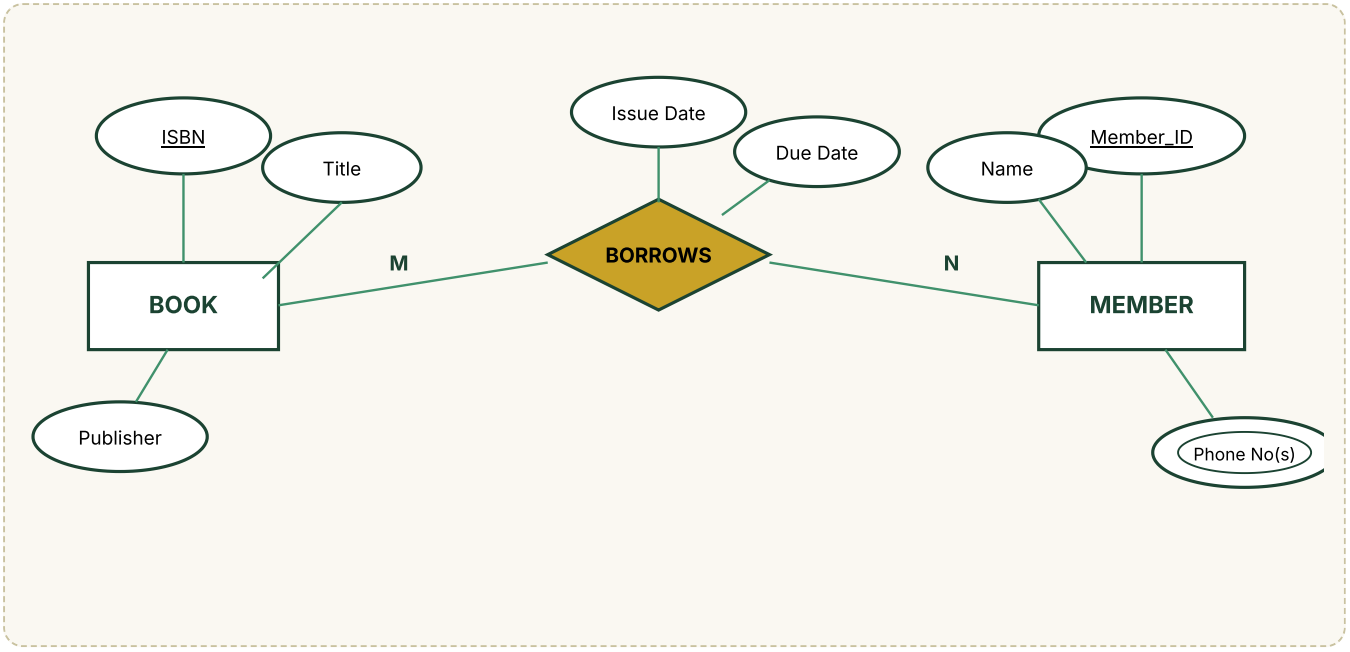
Underline — Key / Primary Key attribute

1, N, M labels on lines = **Cardinality**

1 Library Management System

Many-to-Many | Attribute on Relationship | Multivalued Attribute

A library has **Books** and **Members**. A member can borrow many books over time, and a book (as a title, multiple copies) can be borrowed by many members — a classic **M:N** relationship. We record the *Issue Date* and *Due Date* on the relationship itself, since these facts belong to the act of borrowing, not to the Book or the Member alone.

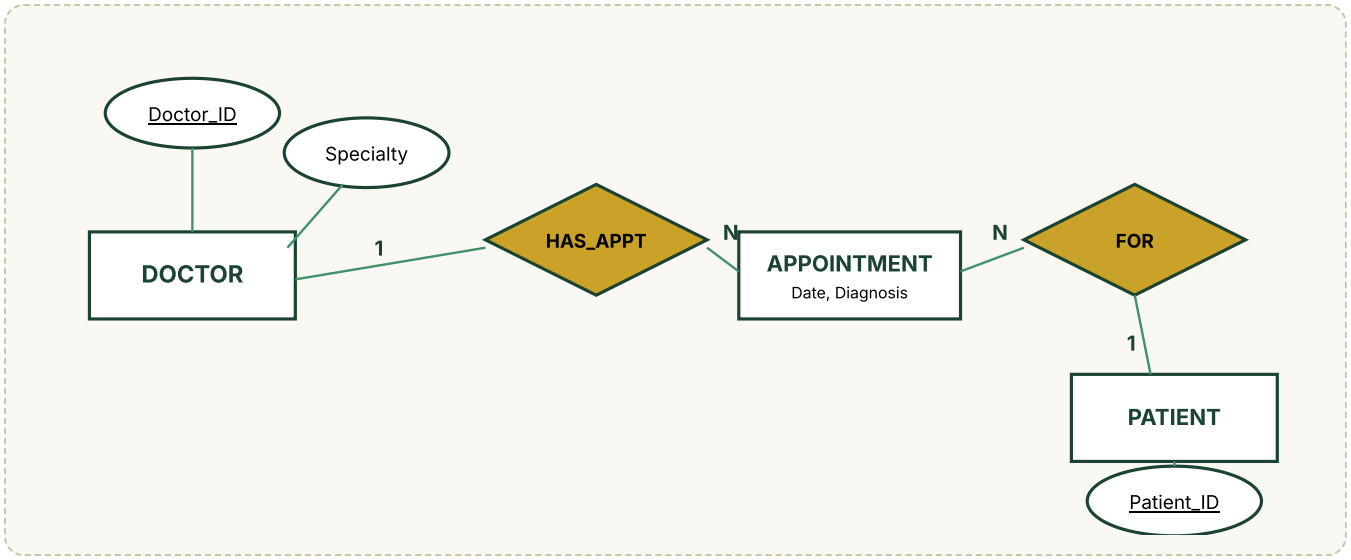


Schema takeaway: BOOK(ISBN, Title, Publisher), MEMBER(Member_ID, Name), and a junction table BORROWS(ISBN, Member_ID, Issue_Date, Due_Date) — because it's M:N, the relationship's own attributes must live in a separate table, not inside Book or Member.

2 Hospital Management System

One-to-Many | Associative Entity | Total Participation

One **Doctor** treats many **Patients**, but each treatment happens through an **Appointment** — patients can also see different doctors on different appointments, so the true link is M:N between Doctor and Patient, resolved through the Appointment entity carrying its own *Date* and *Diagnosis*.

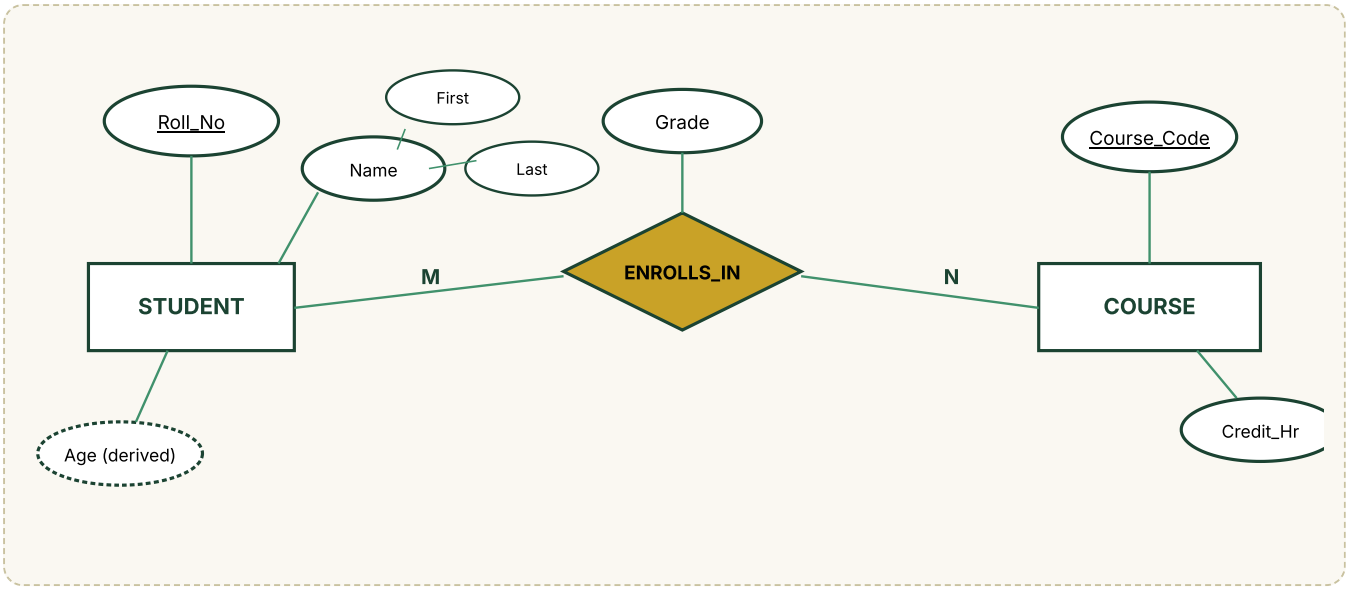


Teaching point: whenever a relationship needs its own attributes *and* both sides are many, model it as an entity (Appointment) rather than a bare M:N relationship — this is the associative entity pattern students will reuse constantly.

3 University Course Enrollment

Many-to-Many | Derived Attribute | Composite Attribute

Students enroll in many **Courses**, and each Course has many Students — M:N, with *Grade* recorded per enrollment. Student's *Age* is derived from Date_of_Birth, and *Name* is composite (First, Last).

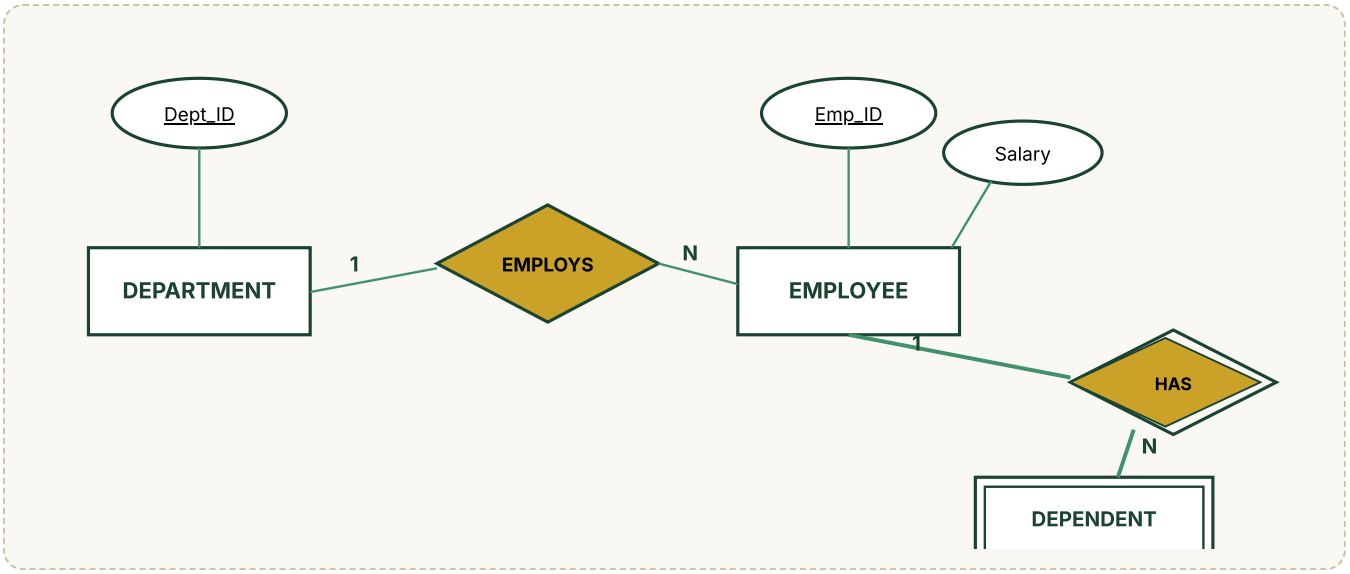


Discussion prompt: ask students why Age should never be stored directly — because Date_of_Birth already implies it, and a stored Age would go stale. This is a good moment to introduce derived attributes and, later, normalization.

4 Company: Department, Employee & Dependents

One-to-Many | Weak Entity | Identifying Relationship | Total Participation

A **Department** has many **Employees** (1:N). Each Employee may have several **Dependents** for insurance purposes — but a Dependent has no meaning or unique ID without its Employee, so it is a **weak entity**, identified only by Dependent_Name combined with the owning Employee's key.

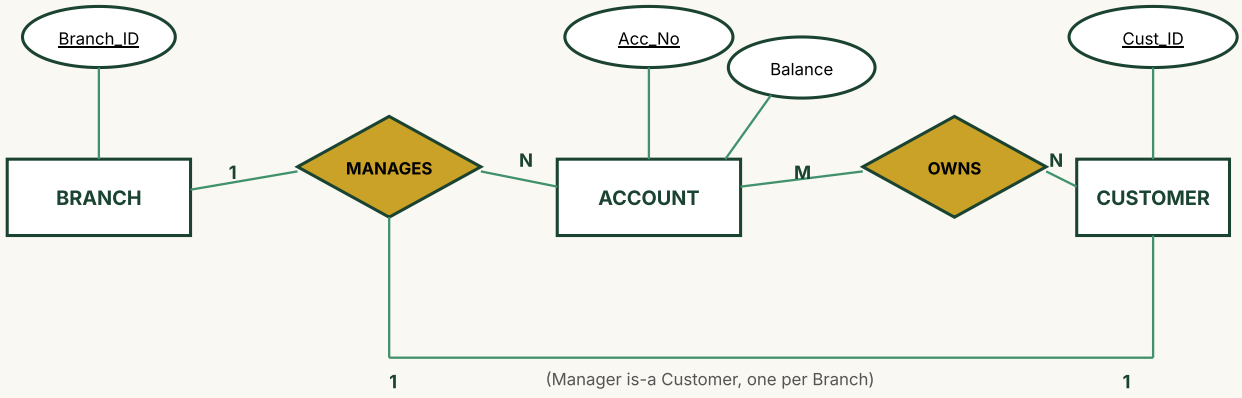


Key idea: the double rectangle (Dependent) and double diamond (HAS) mark a weak entity and its identifying relationship. The thick "1" line shows **total participation** — every Dependent record must be tied to exactly one Employee; it can never exist independently.

5 Bank System

One-to-Many | One-to-One | Multiple Relationships

A **Branch** manages many **Accounts** (1:N). A **Customer** can hold many Accounts, and an Account can (jointly) belong to more than one Customer, so Customer–Account is M:N via "OWNS". Separately, each Branch has exactly one **Manager**, who is a Customer — that's a 1:1 relationship.

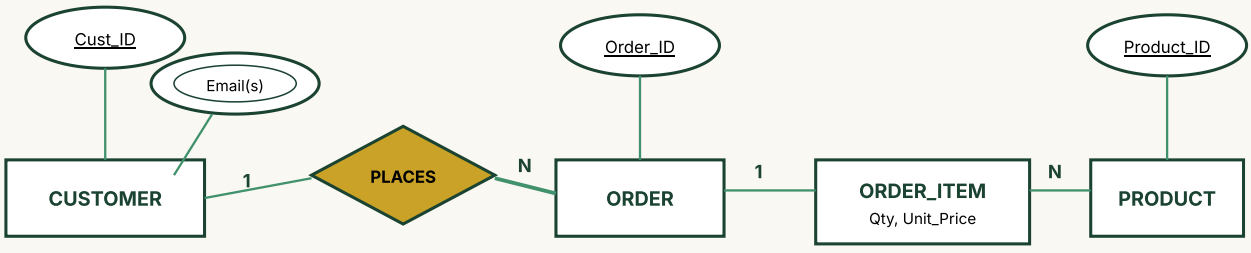


Discussion: good scenario to contrast three cardinalities in one system — 1:N (Branch–Account), M:N (Customer–Account, for joint accounts) and 1:1 (Branch–Manager).

6 E-Commerce Ordering

Associative Entity **Composite Key** **Multivalued Attribute**

A **Customer** places many **Orders** (1:N, total participation on Order). Each Order contains many **Products**, and each Product appears in many Orders — resolved via **ORDER_ITEM**, which carries *Quantity* and *Unit_Price* and whose key is the combination (Order_ID, Product_ID).

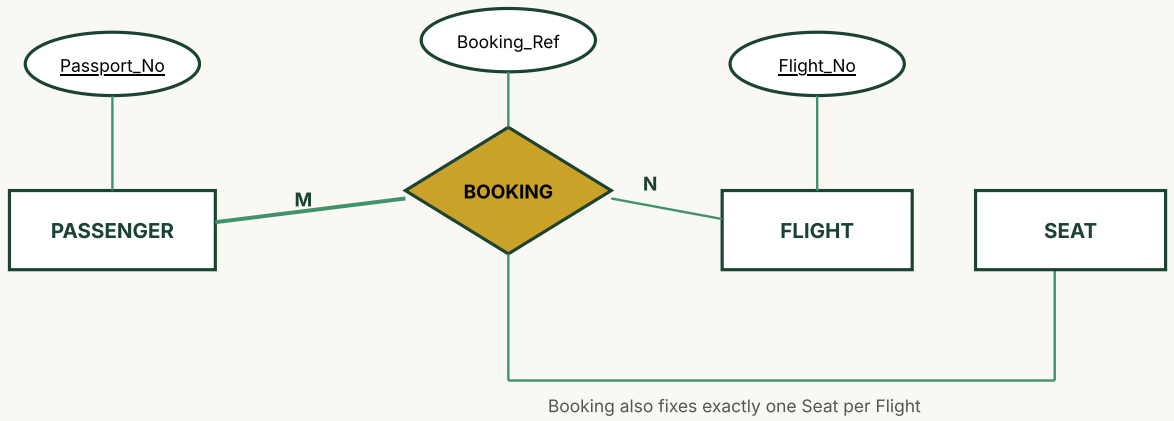


Schema takeaway: ORDER_ITEM(Order_ID, Product_ID, Qty, Unit_Price) — a composite primary key made of two foreign keys is the standard resolution for an M:N relationship that also needs its own data.

7 Airline Reservation System

Many-to-Many | Ternary-style Relationship | Total Participation

A **Passenger** books many **Flights**, and a Flight carries many Passengers — resolved through **BOOKING**, which also references the specific **Seat** assigned, tying three entities together around one transaction (Passenger, Flight, Seat).

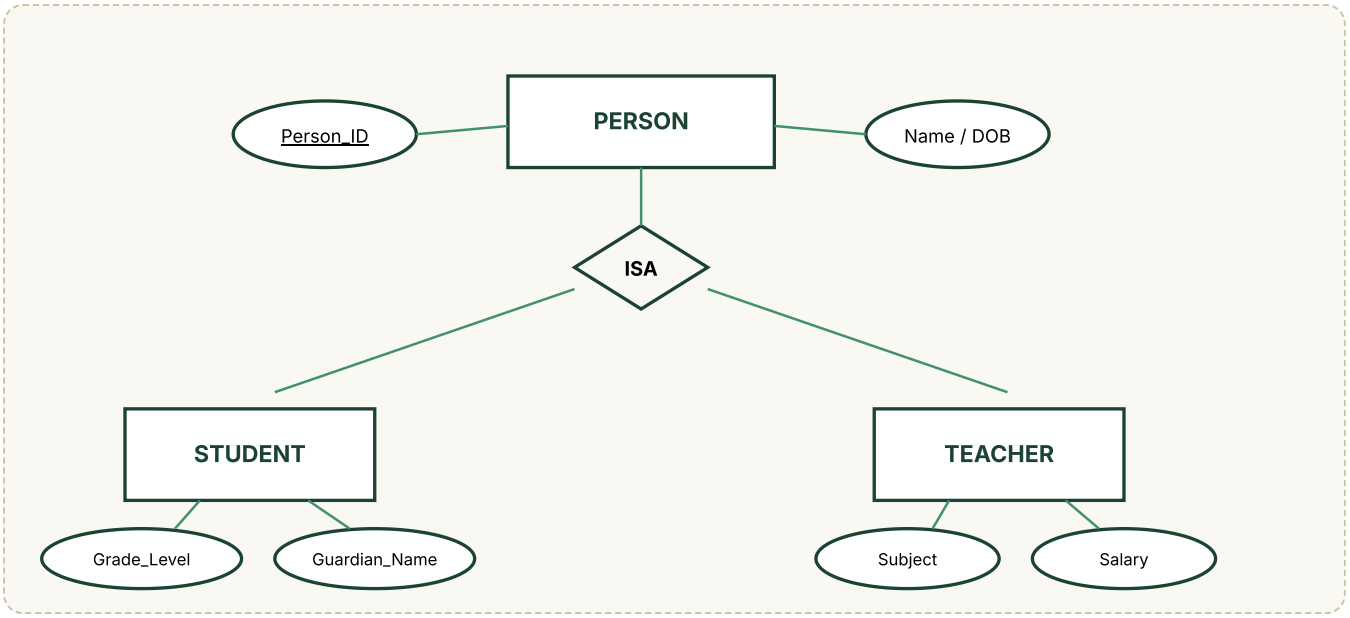


Teaching point: BOOKING is where three entities meet at once — good scenario to introduce the idea that not every relationship is binary, and how a well-designed associative entity keeps the diagram readable instead of drawing one diamond touching three rectangles.

8 School: Generalization / Specialization

ISA Hierarchy | Inheritance | Disjoint Specialization

Both **Teacher** and **Student** are kinds of **Person** — they share Name, Address, Date_of_Birth, but each has attributes the other doesn't (Teacher: Subject, Salary; Student: Grade_Level, Guardian_Name). This is modeled with an ISA triangle: a superclass split into disjoint subclasses.

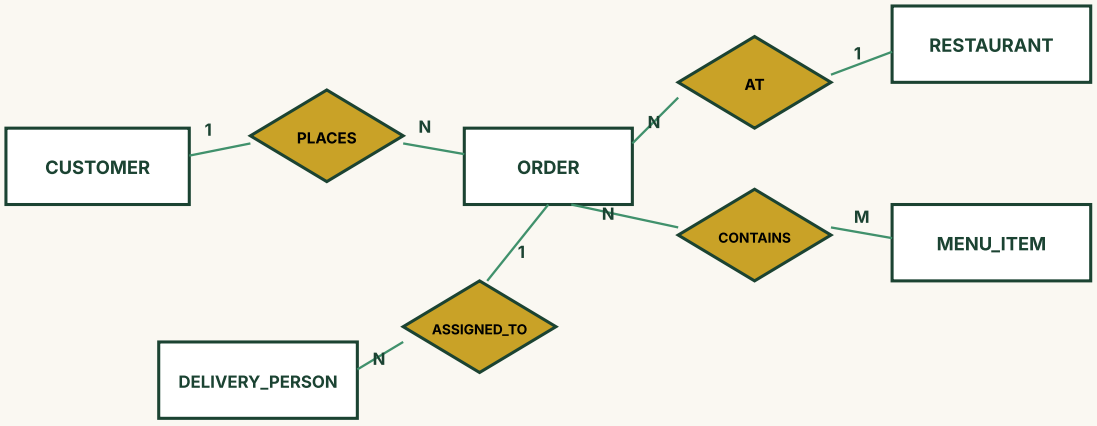


Schema options: discuss with students the three classic mapping strategies — (a) one table per subclass with a shared key, (b) a single Person table with nullable subclass columns, or (c) separate tables entirely with duplicated shared attributes. Each has trade-offs worth a short debate.

9 Online Food Delivery Platform

Multiple Interacting Entities **1:N and M:N combined**

A **Customer** places an **Order** at one **Restaurant** (1:N from Restaurant), the Order contains many **Menu_Items**, and one **Delivery_Person** is assigned to deliver it (1:1 per order, but a delivery person handles many orders over time — 1:N).

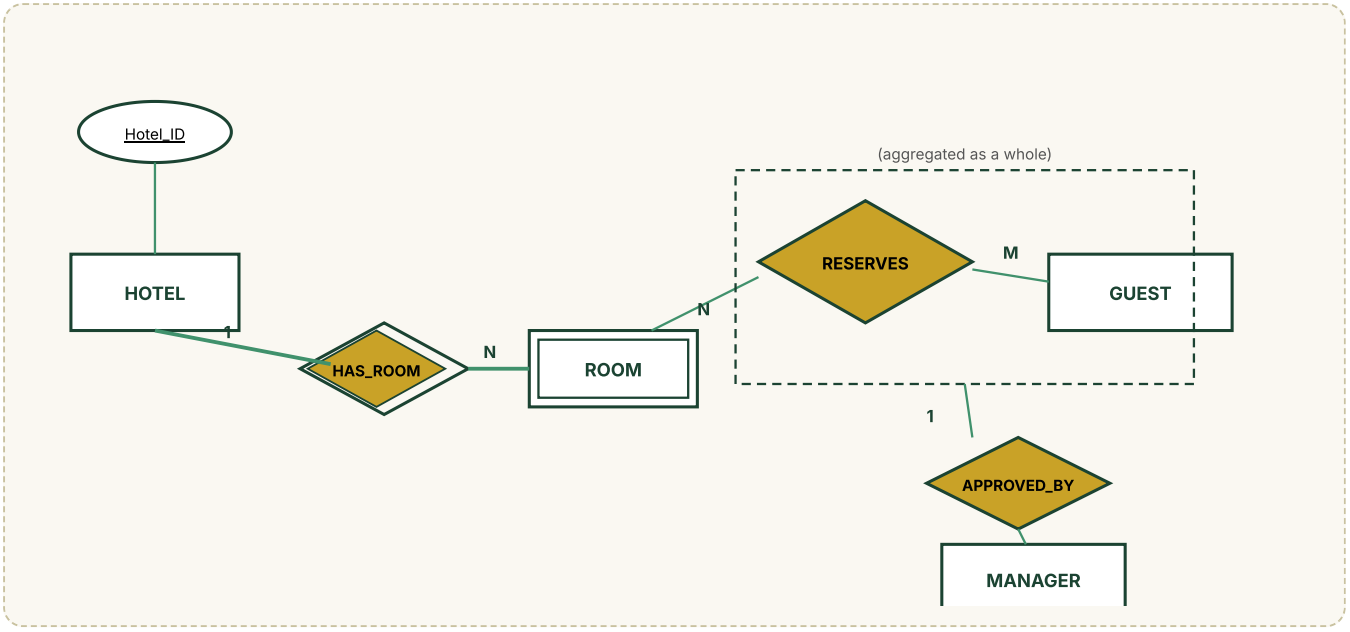


Use as a review scenario — it stacks four entities and four relationships together so students practice reading cardinalities across a busier diagram, similar to what they'll design for a course project.

10 Hotel Reservation System

Weak Entity | Aggregation | Total Participation

Room numbers repeat across hotels, so Room is a weak entity identified only together with its owning **Hotel**. A **Guest** makes a **Reservation** for a Room; separately, that whole Reservation-Room relationship can be "Approved By" a **Manager** — since a relationship (not a plain entity) is being related to Manager, we use **aggregation**.



Why aggregation: "Approved_By" doesn't relate to Room alone or Guest alone — it relates to the *whole act* of reserving a room. Without aggregation students would be tempted to draw a diamond touching another diamond, which Chen's notation does not allow — aggregation is the correct escape hatch.

Teaching Tips & Practice Exercise

Suggested lecture flow

1. Start with Scenario 1 (Library) to introduce entities, attributes, and a simple M:N.
2. Use Scenario 4 (Company/Dependents) to introduce weak entities — students often confuse this with "just another table."
3. Use Scenario 8 (School) to introduce ISA/generalization — a natural bridge into inheritance concepts they may already know from OOP.
4. Close with Scenario 10 (Hotel) to show aggregation, since it requires everything taught before it.

In-class exercise for students

Give students a short paragraph description (e.g. "a gym with trainers, members, and membership plans; a member can freeze their membership for a period") and ask them to independently draw the ER diagram, then compare in pairs. Follow with: "convert your diagram into relational tables" to connect the ER model directly to the next topic — **mapping ER to relational schema**.